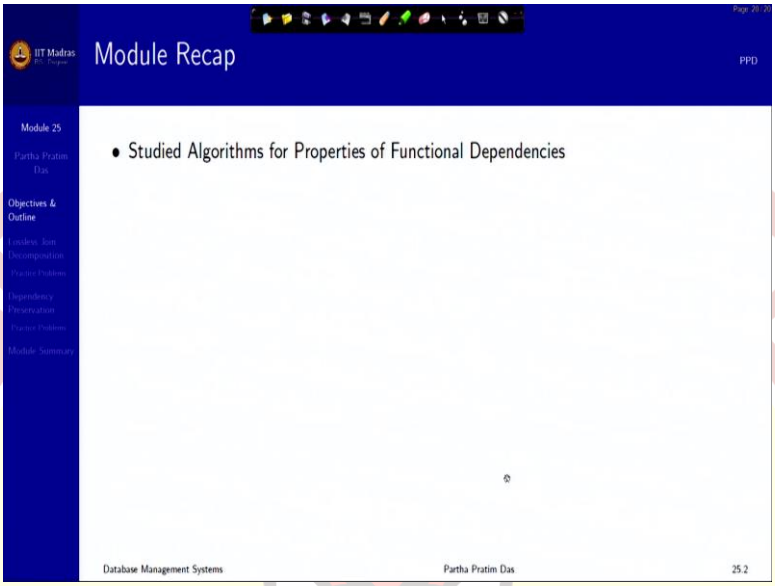
IIT Madras

ONLINE DEGREE

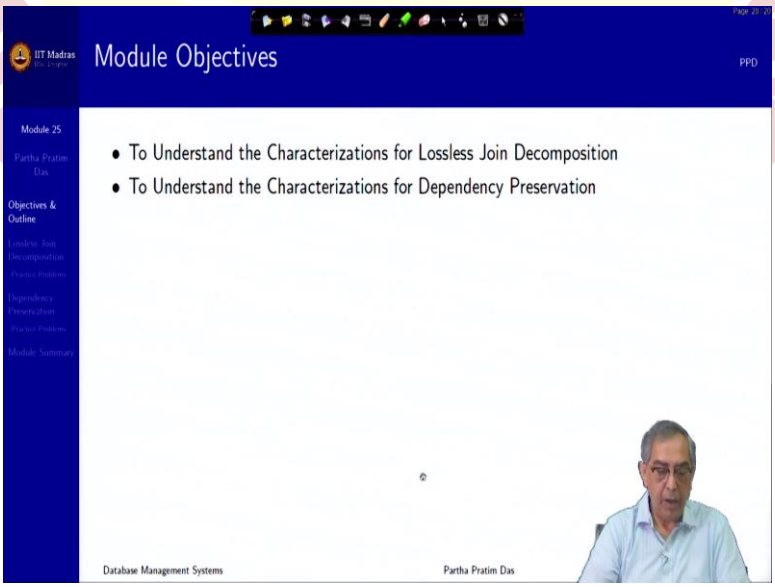
Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Relational Database Design/5

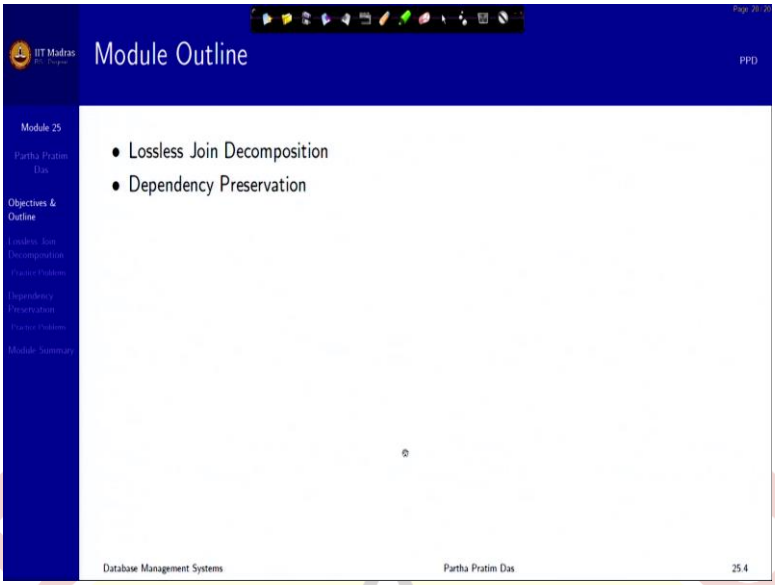
(Refer Slide Time: 0:30)

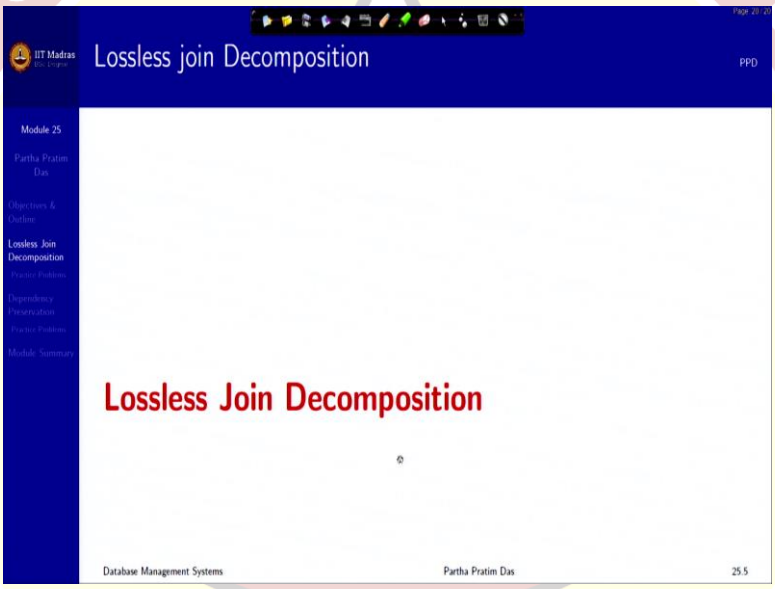


Welcome to Module 25 of Database Management Systems course, in IIT Madras online BSc program. We have been discussing about relational database design and it is based on depths of dependency theories and normalisation, we will talk, we have been talking about that, and in the last module, we have done a number of very important algorithms for functional dependencies.

(Refer Slide Time: 0:49)

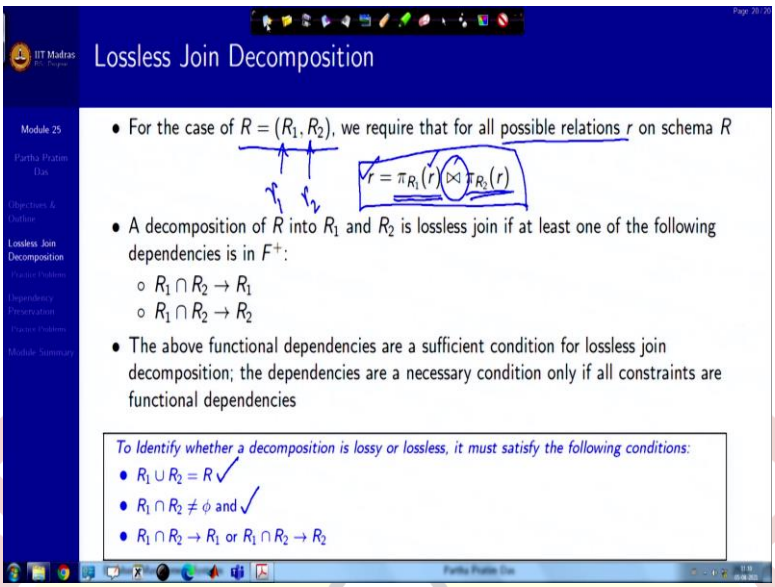


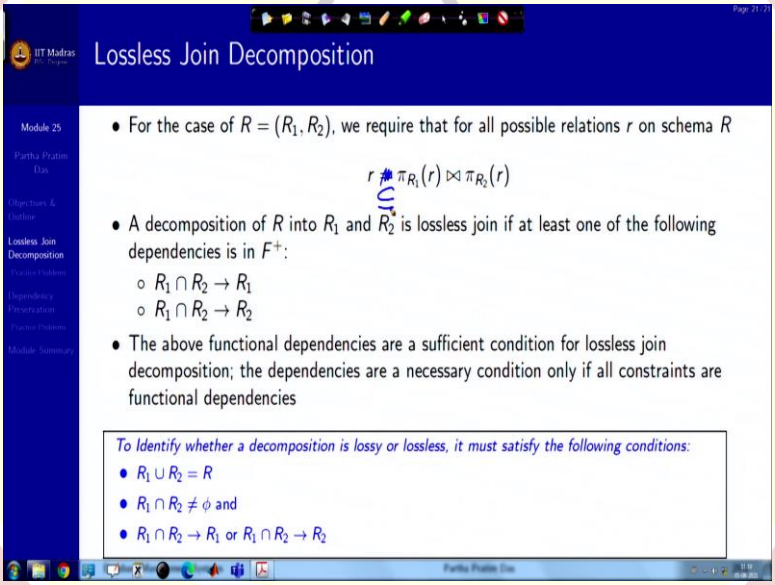


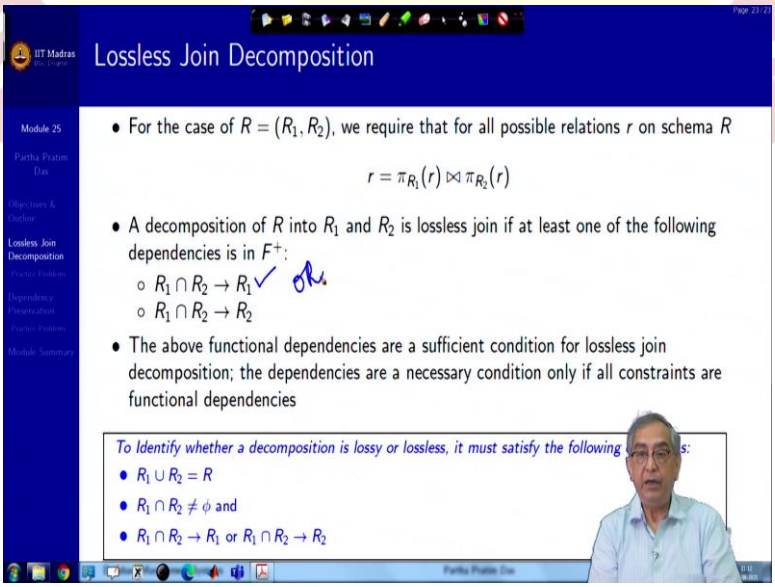


In the current module, what we will try to do is to characterise and understand the depths of two very important properties that decompositions of relations need to maintain, which is lossless join decomposition and dependency preserving decomposition. So, let me start with the lossless join decomposition.

(Refer Slide Time: 1:19)







So, we have in an earlier module discussed about the lossless join decomposition in brief and you will also find the same definition there, may be written in different notation. But, the basic point of a lossless join decomposition is suppose I have a relational schema R which I have divided into two relational schemas R1 and R2.

Now, the question is instead of maintaining the instances of R, I am maintaining the instances of R1 and R2. So, what I need? I need the basic requirement that for any possible instance of R, I should be able to reconstruct that instance from R1 and R2. If I cannot do that, then I have lost information.

So, this is very important to note that this must hold for all possible relation, all possible instances. So, if I have to do that, then what are the basic things I will need? Naturally one thing condition that I will need is that the union of R1 and R2 covers all attributes of R, otherwise obviously, I cannot get back to the information. The intersection of R1 and R2 must be non-null, there must be some common attribute because if there is no common attribute, how do I correlate the segmented information.

And there is a third condition, but what does this lossless join necessarily mean, is the statement that, if you take any instance of R relational schema R written here in capital, uppercase and projected on the set of attributes R1. So, this is what you will maintain as an instance R1 of schema uppercase R1, and if you take a projection of R with respect to R2 you get an instance R2 with respect to the relational schema uppercase R2. You have these two.

If you take a natural join between them, natural join with they have assume that there is some common attributes. So, based on that you can take a natural join, you must get this back It is easy to show that in general, in general this join will give you something which is equal or more, more tuples may come into it. Now, if it does then your decomposition is not lossless, it is lossy. So, that is what we want to ensure.

Now, how to ensure that? Just think about what are we joining on? We are joining on common attributes, like what are the common attributes? Common attributes are R1 intersection R2. These are the attributes it could be one attribute or it could be several, but this is a set of common attributes.

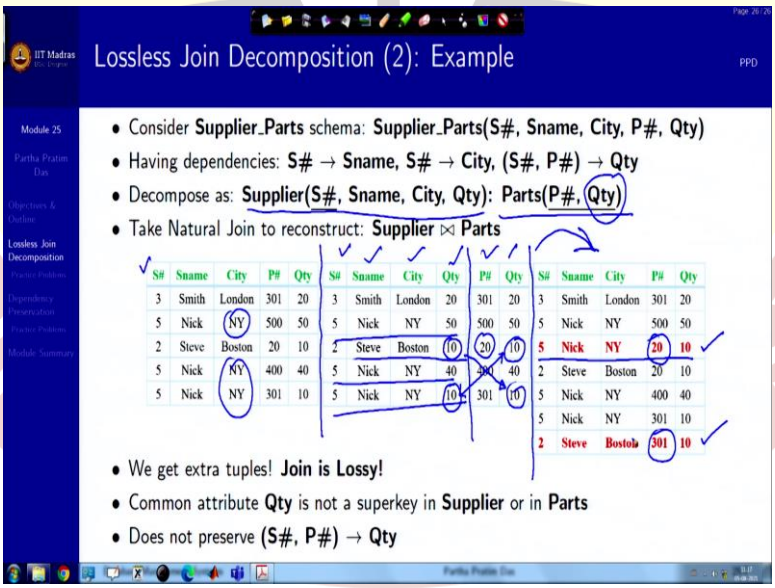
Now, the common attributes if you have non-uniqueness on both R1 and R2 with respect to this common attribute then obviously different combinations which you were not there originally will come out. But what if say this set of common attributes uniquely determines

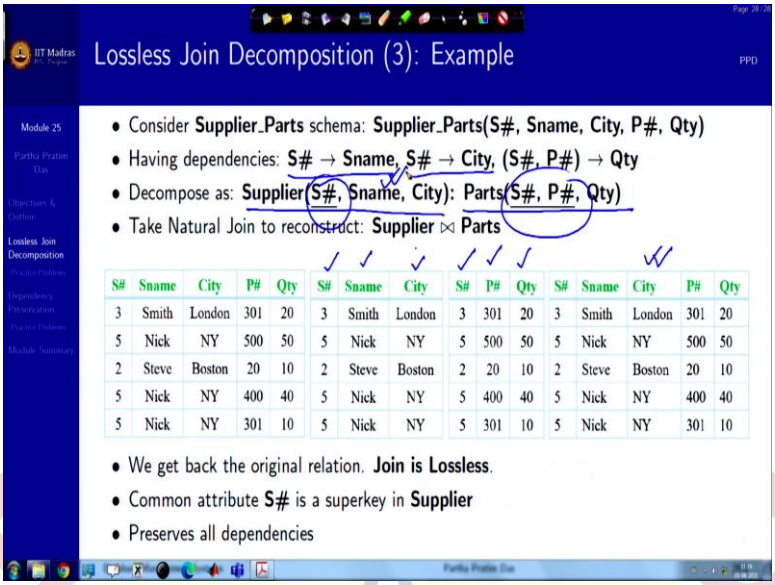
R1 that means it is a key in R1, then what will happen? You ensure that on this common attribute there is only one instance, one tuple that exists in R1 for each value on this R1 intersection R2, it gives key. So, it uniquifies, so, on those attributes you will always have unique set of values.

So, that will pick up the corresponding values here on the, on the R2 side. So, you cannot have non-existent combinations coming. Now, you do not need this on both sides, just one side is okay. So, you can have this that is R1 intersection R2 is a key in R1 or R1 intersection R2 is a key in R2. It does not impact if it is a key in both but that is not required. The necessary condition is the rather the sufficient condition is that the intersection set must be a key in one of the two relations into which you have decomposed.

So, the union of attributes should match the total set, intersection must be non-null and the intersection set of attributes must be key in one of the decomposed relations at least. So, that is the sufficient condition to ensure that you have lossless join. Why and how it will become a necessary condition and all, we will discuss later on. But I am just not going through the proof, I am just giving you the intuition behind this.

(Refer Slide Time: 7:06)





So, we can check up on example, say I have a supplier schema which has supplier parts relation, supplier ID, supplier name, the city where the supply belongs, the part number supplied and the quantity supplied. Naturally the dependencies supplier ID will determine supplier name, supplier ID will also determine the city, the city in which the supplier is located and how do you get the quantity? We will get the quantity from the supplier ID and part number pair, only that, with that you can get this.

Now suppose so, this is if you, if you, if you see 1 2 3 4 5 so, this is it is up to this if you are not seeing the boundaries well, these are the boundaries. So, this is the original relation and instance. And we are trying to, naturally this has has this will have possible redundancies here. For example, Nick and Nick's city is occurring multiple times and so on.

Now we want to decompose this and this is what we have done. In one relation we have put the supplier, supplier ID, name, city and quantity. In other we have put product ID and quantity. This is a decomposition we have done. Now for this instance if I reconstruct by natural join, there are five tuples originally and now we have seven.

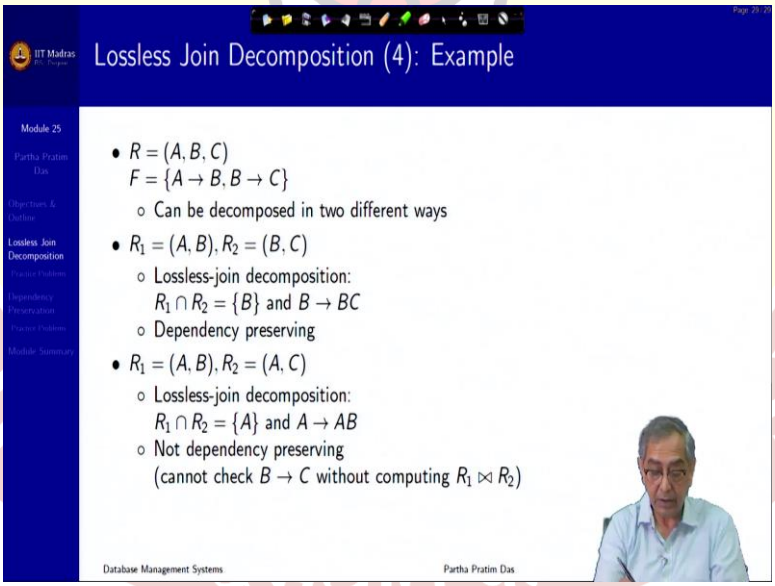
These two in red have come up which were actually not there. So, this tuple which was not present has come up here, you can see. You had Nick here, you had Nick here, you had Nick here, the common attribute is quantity. Now there is quantity 10 here, there is quantity 10 here. So, when you do the join, you not only get Nick NY 10 with your 301 but you also get Nick getting combined based on this with originally the Steve's order, Steve's supply, we are getting combined like this and that is the reason you get a 20 here in the result.

Similarly, Steve's get combined here. So, you get a 301 in the result. So, you have additional tuples coming in, the join is necessarily lossy, why did we go wrong is that we have not, we have ensured the union rule, we have ensured the intersection rule, but quantity is not a key in either of the decomposed relation. It is also a fact that it does not preserve the, this particular dependency.

But if we decompose this differently, if we decompose this in this way that is S name, S ID, S name and city, S ID, product ID and quantity, then we will, you get a perfect reconstruction in this. Why? Why do you get that? Because now, the common attribute is S ID, S #, the supplier ID.

And now the supplier ID is a super key in the supplier because it determines S name, it determines the city so, supplier ID is a super key here. It is not a super key here, because here it is the pair but is a super key in this. Therefore, it the intersection set of attributes is a super key here which ensures that in the supplier S name city in this you have unique uniqueness in terms of the representation and therefore, you have ensured that the joint is lossless.

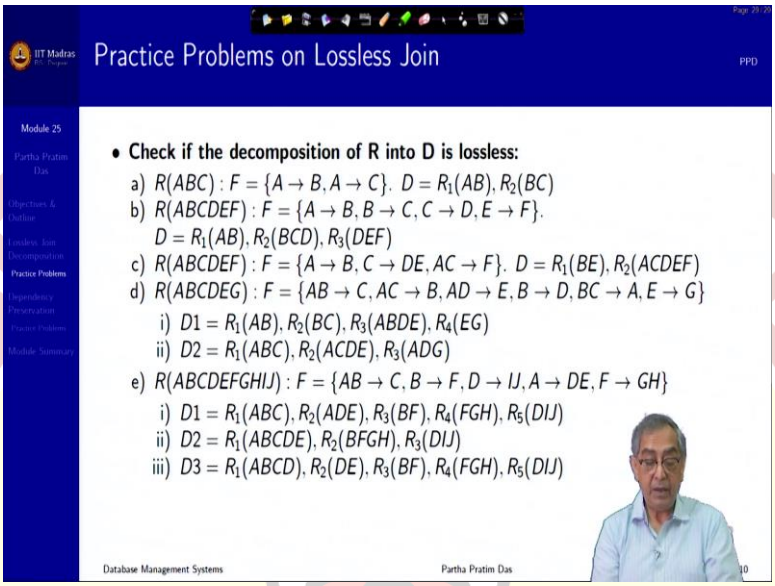
(Refer Slide Time: 12:41)



Now, you can these are, these are different example A determines B, B determines C can be decomposed as AB and BC which is a lossless join easily, because the intersection B is a key in the, in the BC decomposed relation. It is also dependency preserving because you can check A determines B in the first one B determines C in the second one it preserves. You can decompose it as AB and AC.

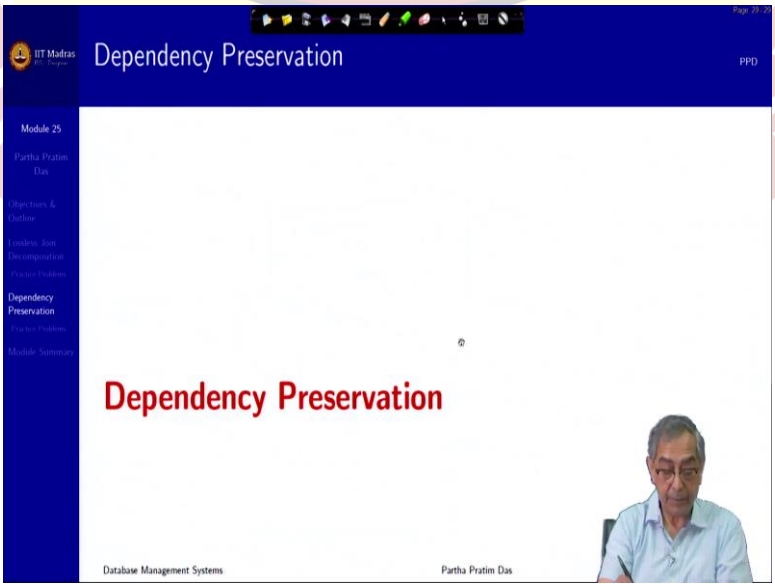
If you, do it as AB and AC, even then it is a lossless join decomposition because in AB and in AC both A is a key. So, this is a lossless join decomposition. But this is not dependency preserving. Why? Because the dependency BC, B determine C cannot be checked either in AB or in AC because both attributes are not there. We will talk more about this.

(Refer Slide Time: 13:57)



So, there is a couple of practice problems on lossless joint you have to work through these different functional dependencies for different relational schemas are given with the decomposition and you have to check and tell whether it is lossless or not applying the rules just to get practised.

(Refer Slide Time: 14:18)



Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
  - A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

  - If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive

Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if

- $F_1 \cup F_2 \equiv F$ {Decomposition Preserving Dependency}
- If $F_1 \cup F_2 \subset F$ {Decomposition NOT Preserving Dependency} and
- $F_1 \cup F_2 \supset F$ {this is not possible}

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
  - A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

  - If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive

Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if

- $F_1 \cup F_2 \equiv F$ {Decomposition Preserving Dependency}
- If $F_1 \cup F_2 \subset F$ {Decomposition NOT Preserving Dependency} and
- $F_1 \cup F_2 \supset F$ {this is not possible}

$(F_1 \cup F_2)^+ = F^+$

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
  - A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

  - If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive

Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if

- $F_1 \cup F_2 \equiv F$ {Decomposition Preserving Dependency}
- If $F_1 \cup F_2 \subset F$ {Decomposition NOT Preserving Dependency} and
- $F_1 \cup F_2 \supset F$ {this is not possible}

Now the dependency preservation the other rule. What you need is, all dependencies must be preserved which means that I must be able to check all dependencies in the decomposed relations. I can always take a join, take the entire table and check for the all functional dependencies but that is enormously time consuming this will not work.

So, it must be possible to check that each and every dependency holds in the through the satisfaction of functional dependencies in the decompose sets. So, if we say that if I is a set of dependencies in F+, it may not be F, F+ the closure, if Fi is a set of dependencies in F+ that contains only attributes in Ri. So, kind of, you are just filtering out attributes in Ri, then what it, what it signifies that Fi can be fully checked on Ri, because it has attributes only.

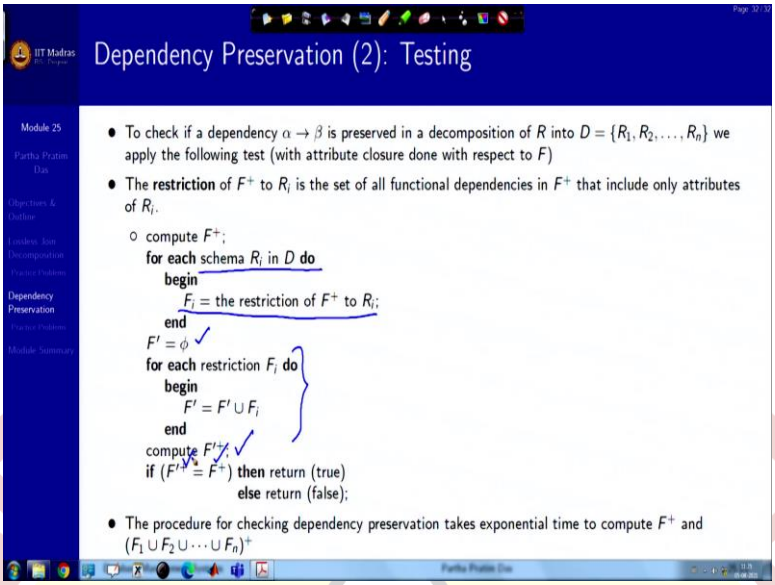
And Fi also is a largest set of functional dependencies, logically implied by F that can be checked on Ri, because Fi you have taken as a subset of F+. So, now, if you consider, if there are, if you have decomposed into n different relations, different schemas, then you will have a F1, F2, Fn. So, these are the sets of functional dependencies that you can check without taking a join on the individual corresponding R1 not to Rn.

So, this is the whole set you can check. So, what is the set that is implied by this check, is a closure. So, if you check F1, whole of F1, F2, Fn in the corresponding relational schemas, then in totality, what you are checking is the closure of their union set. If that is equivalent to F, that is if that is equal to the closure of F, then all dependencies are preserved. But if it is a subset of F+ then there is some dependency which is not being preserved. That is the basic idea.

So, we will say that naturally, this is the general notion and definition, we will again work with a just a binary decomposition. So, you have R1, and R2 having functional dependencies F1 and F2. And that decomposition R is said to be preserving, if, F1 union F2 is equal to F, is equivalent to F not equal to, equivalent to, which means actually, what you mean is, F1 union F2 that equivalent to F, then it is dependency preserving.

If they are equivalent, if they are covered, if F covers f1 union f2, but F1 union F2 does not cover F, then it is not dependency preserving, obviously some dependencies on. The other side that F1 and F2 covers F but F does not cover F1 union F2 is not possible, because you have started with a subset from F+ both in F1 as well as in F2. So, these cases cannot arise. So, we have just two cases, this is dependency preserving, this dependency not being preserved.

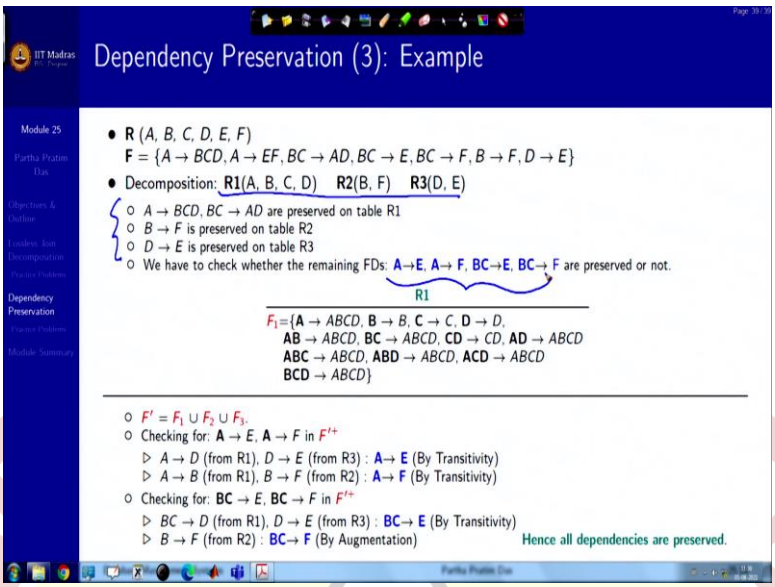
(Refer Slide Time: 18:20)



So how do you compute that this is a naive algorithm, but a clear one. So, to compute F+, you begin with, for each relational schema Ri, you need to compute Fi that is from F+ you have to project it onto take out attributes which have only Ri and you have the different Fi's. Then you want to basically compute the closure for this union set.

So, in the second what you do, you take the union of all Fi's into F prime, compute the closure, you have the closure of F, check if they are equal. If they are equal, then you return true otherwise you return false. Very simple straightforward algorithm only hitch that it will have is checking the procedure for checking dependency preservation takes exponential time to compute, because you are having to compute a closure which is exponential time, but we have some algorithm.

(Refer Slide Time: 19:30)



So, let us look at one example of to understand what how this is going. So, this is the set of attributes, these are the functional dependencies and these are the decompositions. Now, if I am making more of observations that obviously this has A, B, C, D. So, I can say that this one will be preserved in R1 and BC AD, this will also be preserved on r1 R1, that is the first example. B determines F, we have BF, so, on R2 we will have B determines F. D determines E can be checked R3. So, these are directly there.

So, we have to check for the remaining whether they are preserved or not. So, just, humanly just making the task a little easier. Now, we compute F+ and take out the dependencies which are containing attributes of R1 that is ABCD and that is this set A determines ABCD, B determines B, C determines C, D determines D, you have done a closure, so, you get all these kind of trivial dependencies also AB determine BC determining ABCDM, CD determining, so, pairwise attributes, then triplets of attributes.

So, this is the entire F1 set, these are the functional dependencies that will get checked on R1 through whatever we can check in R1. So, similarly, you can do for R2, for R3. Now, let us say, let us consider the first one, A determines E. Now, what do I have? I have A determines ABCD. So, which means, I have A determines D, if A determines ABCD, A determines D.

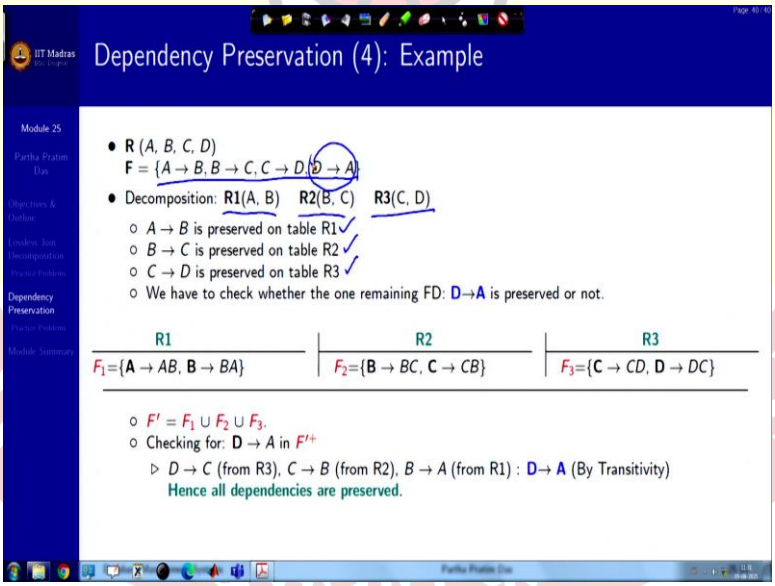
So, I have A determines D from R1, R1 will make sure that A determines D. We have D determines E from R3, we have already seen. So, R1 ensures that A determines D, R3 ensures that D determines E, if these two are ensured then by transitivity A determines E is ensured. So, this first case is good, A determines F from R1 again I have A determines B from R2, I have B determines F by transitivity I have A determines F. Very simple.

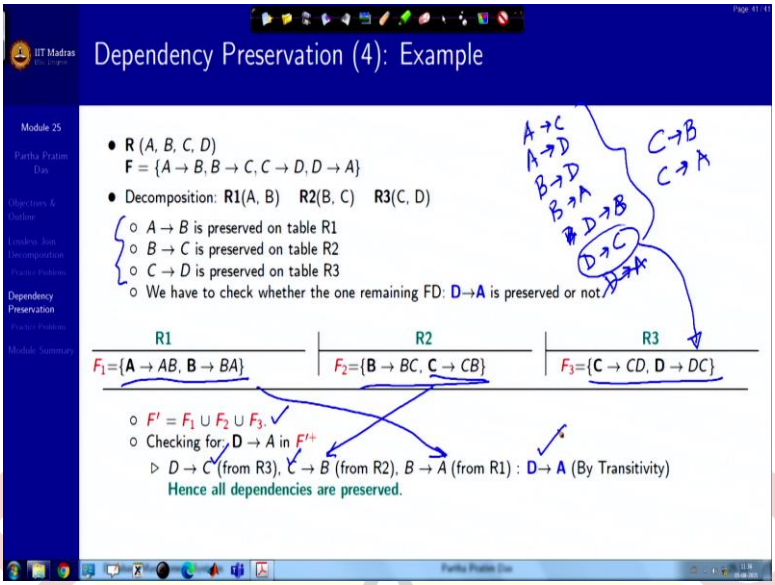
So, this is done, this is done. I have to check for, so, BC determines E. Now, what do I have from R1? From R1 I have BC determines AD, if it determines AD then it determines D. So, BC determines D. What do I have from R3? Is D determines C by transitivity I have BC determines C.

Now, obviously, this is not the way the machine will work, the machine will work through the algorithm. But this is how we are just doing by observation to understand that how the preservation is happening even though these these dependencies may have attributes which are distributed over two or more different decomposed relational schemas. The last one left is BC determines F.

Now, you have B determines F in R2, if B determines F by augmentation BC determines F. So, as we see that through this decomposition, we directly can check for these four functional dependencies. But having checked them, we actually ensure that these functional dependencies will also hold and therefore, this decomposition is called a dependency preserving decomposition.

(Refer Slide Time: 24:36)





One more quick example is a there are four, there are four dependencies. Now you have projected on three, you have composed into three relationships, naturally, three of these dependencies are can be checked directly, obvious to see, this one cannot be checked, because there is no relationship, relational schema, where A and D are occurring together.

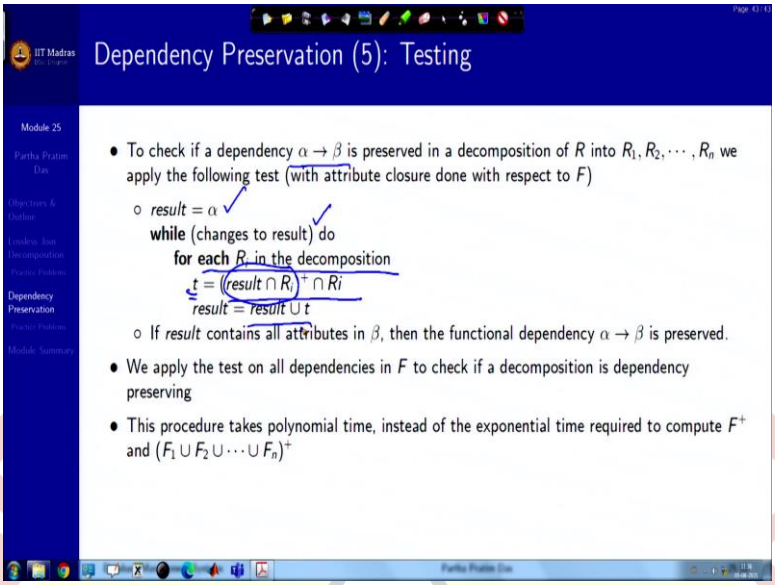
Now, you have to, what insight do you get in here? You can easily see that these are kind of circular dependency. So, if A determines B, B determines C, C determines D, D determines A. So, what do I get? I get A determines C, I get A determines D, I get B determines D, I get B determines A, I get D determines, I get D determines A, A is there, B because D determines A, A determines B, D determines C, I get D determines A.

So, if you see actually here, everybody actually determines everyone else. So, if I can ensure the compliance with three of these dependencies, then I will comply the fourth one. Just to formally see in R1, I have AB. So, this is a F1 set, A determines AB, B determines AB. Similarly, in R2, similarly in R3. So, if you do the union, you have all of this together.

Now, so, I have if you see here, this one that I had has gone in, so, D determine C, it has come in in R3 in the, in the through the closure. So, D determine C is from the closure, C determines B, I did not compute C here, C determines B, C determines A, C determines D, b comes in from here and B determines A comes in from here, that the missing part of the, the reverse part of the cycle.

So, D determines C, C determines B, B determines A transitivity twice D determines A. So, even when apparently, it might look that some decompositions may violate the preservation, it actually does not, it can, so, through the closure process it could do that.

(Refer Slide Time: 28:10)



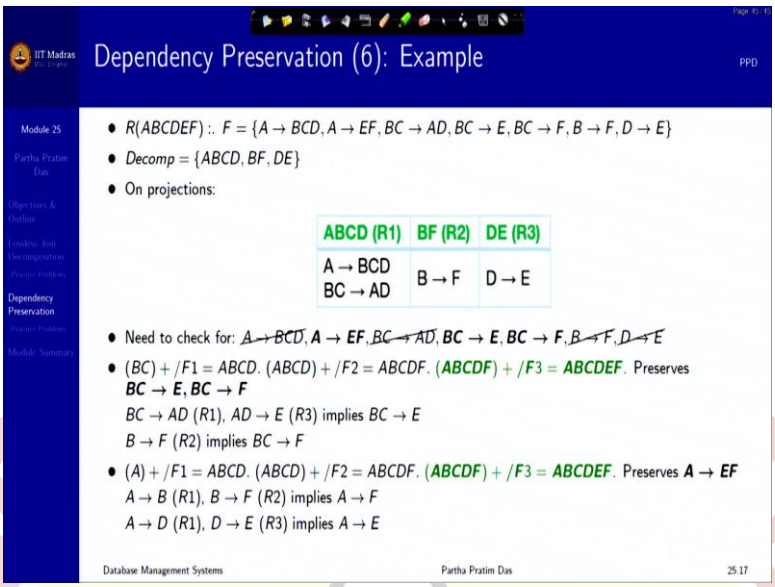
Now, it is let us look at, so, what we actually want? The way we have seen, we have not done the actual all this total closure and all, what we have tried to do is does the question is does this functional dependency which is missing, is it implied by the set that I have. So, how do I find that, again closure of attributes. So, we will try to do that, we will try to.

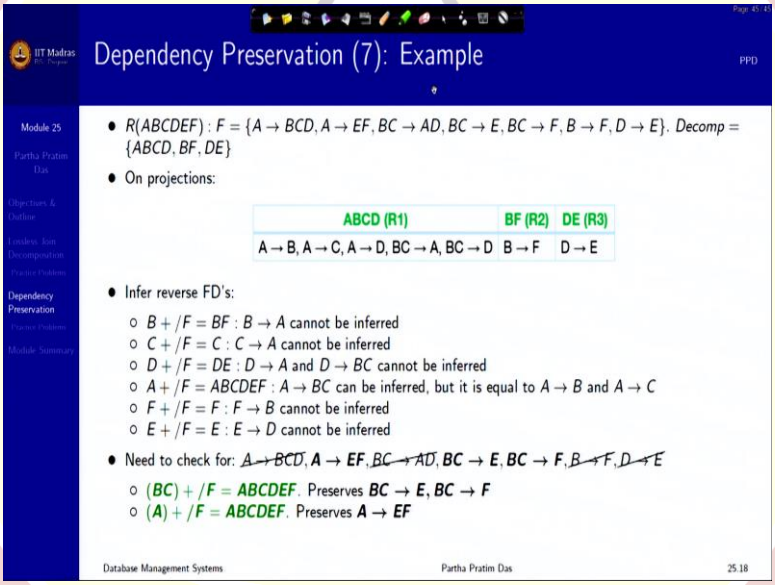
So, I said I want to check if alpha determines beta is preserved, the decomposition of R1 to Rn. So, by for the, for the closure of alpha I set result to alpha as long as the result does not change, I keep on doing this. For each decomposition, I take out attributes only in Ri, compute the closure of the attributes with respect to F and again take the intersection with Ri. Why do I take first the intersection?

I take the first intersection because that is what I can check. Why do I take, then I take the closure. Why do I take the second intersection? Because that is a result that I can use. And what we have got I add to the result, I keep on doing this. So, what I get is at the end when it ends, if the result contains beta, result is a superset of beta then I know that alpha determining beta can be checked can be preserved in this decomposition.

Very simple, so, you can see that the algorithm to compute the closure of attributes is very, very powerful in doing things. So, here is the whole procedure is polynomial time, because closure computations polynomial time and instead of exponential time we saw if we wanted to actually compute the whole set of closure of functional dependencies. So, that gives us a very practical way to compute the dependency preservation.

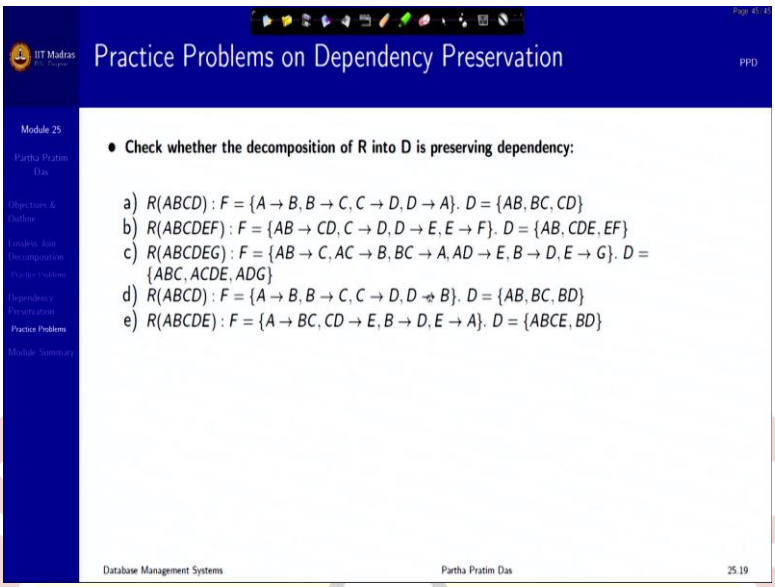
(Refer Slide Time: 30:34)

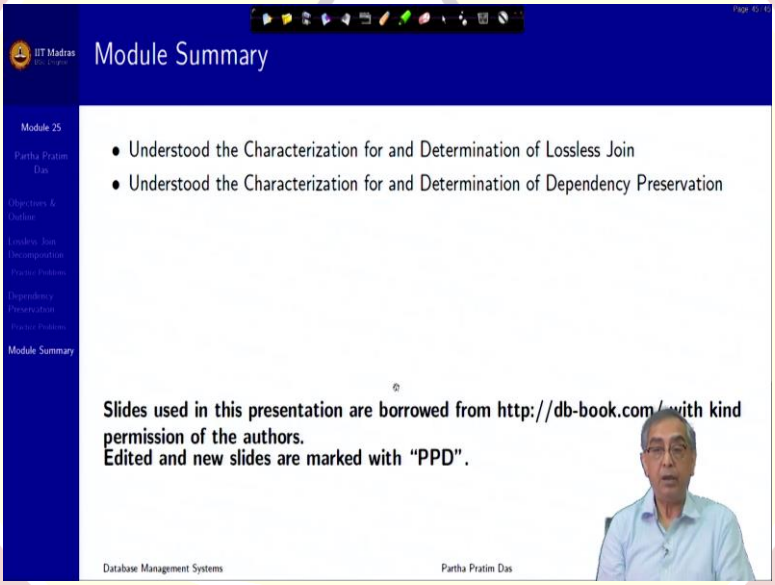




In the remaining two slides, we have, I have given worked out examples, actually the previous examples to working through the closure of attributes I will not go through them one by one here, I will just leave it for your study, you can work out and check that the same thing as we were doing with the closure of functional dependencies can be done very well with the closure of attributes.

(Refer Slide Time: 31:09)





And to end here are some practice problems for you, which you should obviously practice before we move on to the next module. Because unless you have a good hold on the algorithms of functional dependencies, I gave in the last problems, I gave in the last module and on lossless join and dependency preservation unless you have a good hold on that normalisation and normal forms will be difficult to work with. So, this is what we have learnt in this module. Thank you very much for your attention and we will meet in the next module. Continuing on this.